

Provenance-Conditional Hyperbolic Graph Learning

on Traced Formal-Math Hierarchies: Pipeline, Protocol, Diagnostics, and Evidence Boundaries

Anonymous Author(s)

Anonymous Institution

Abstract. We present a reproducible pipeline, evaluation protocol, diagnostics framework, training alignment correction, and an ancestor explanation tool for studying graph representation learning on real traced Lean/Mathlib hierarchy graphs. Our central finding is provenance-conditional: on the full source graph (containing both `extends` and `instance_of` edges), GCN remains stronger; HGNCN’s advantage emerges only on the explicit hierarchy layer (`extends` edges alone), where genuine depth exists, and grows monotonically with ancestor chain depth (+0.03 at `hop_2` to +0.27 at `hop_4_plus`).

Contributions: (C1) a version-locked extraction pipeline from traced repositories to relation-aware subgraphs; (C2) a grouped multi-positive ancestor retrieval protocol with query-level splits and hop-bucket analysis; (C3) a structural diagnostics framework with heuristic classification thresholds; (C4) the first demonstration that edge provenance composition—not graph size or capacity—determines which geometry is favored; (C5) migration from binary edge classification to query-grouped retrieval training. We validate through 60 provenance-aware seed sweeps (zero failures) across two candidate graphs and three provenance splits, and demonstrate downstream proof-engineering relevance via an ancestor explanation tool.

Evidence is from reviewed single-environment runs; the finding is established on two candidate graphs (133 and 253 nodes).

1 Introduction

1.1 Motivation

Interactive theorem provers such as Lean, Coq, and Isabelle organize mathematical knowledge in rich hierarchical structures. In Lean/Mathlib, declarations (definitions, theorems, type classes) are connected by `extends` edges (explicit inheritance), `instance_of` edges (synthesized type-class instance registrations), and `uses` edges (referential dependencies). These hierarchy graphs are natural targets for graph representation learning: the hierarchical structure encodes

mathematical knowledge at multiple levels of abstraction, and navigating this hierarchy efficiently could support proof search, declaration recommendation, and mathematical knowledge exploration.

Hyperbolic graph neural networks (HGCN) offer a theoretically appealing inductive bias for such data: hyperbolic space can embed tree-like structures with exponentially lower distortion than Euclidean space, and the distance scaling properties of hyperbolic geometry are well-suited to representing ancestor–descendant relationships. However, empirical evidence for hyperbolic advantage on real formal-math graphs has been mixed. Prior work has reported that HGCN does not consistently outperform Euclidean baselines (GCN) on traced Lean hierarchy graphs, raising the question: under what structural conditions does the hyperbolic inductive bias actually help?

1.2 Background

Lean/Mathlib hierarchy semantics. Lean 4 and its mathematical library Mathlib organize knowledge through a type-class hierarchy. Declarations (definitions, theorems, instances) are connected by two primary hierarchy edge types: **extends** edges encode explicit inheritance (e.g., `CommRing` extends `Ring` extends `Semiring`), creating chains of increasing abstraction with genuine depth and branching; **instance_of** edges register synthesized type-class instances (e.g., `Int.instCommRing` is an instance of `CommRing`), connecting concrete implementations to abstract interfaces without creating inheritance chains. A third edge type, **uses**, captures referential dependencies but does not participate in the hierarchy layer. This distinction between explicit and synthesized hierarchy edges is Lean-specific and has not been systematically controlled for in prior graph-learning experiments on formal-math data.

Hyperbolic graph neural networks. Hyperbolic space offers exponentially lower distortion for embedding tree-like structures compared to Euclidean space. HGCN (Hyperbolic Graph Convolutional Networks) leverages this by performing message passing in hyperbolic geometry, with a learnable curvature parameter. The theoretical motivation is that ancestor–descendant relationships in hierarchies should be more naturally captured by hyperbolic distance scaling. However, the practical benefit depends on the actual tree-likeness of the input graph: on flat or shallow structures, the additional representational capacity of hyperbolic geometry can be a liability rather than an advantage.

Formal-math graph tooling. Prior work on graph-based approaches to formal mathematics has focused on premise retrieval, proof search guidance, and declaration recommendation. These efforts typically treat the graph as a uniform structure without distinguishing edge provenance. Our work differs in focus: rather than proposing a new model architecture, we provide the methodological infrastructure—pipeline, protocol, diagnostics—needed to systematically determine *under which structural conditions* different representation geometries are effective.

1.3 The Problem

We identify two categories of confounds that have obscured the answer to this question:

Protocol confounds. The standard evaluation protocol for ancestor retrieval in formal-math graphs has historically treated the task as single-positive ranking—finding one true ancestor among candidates. In reality, the task is multi-positive: a given (**declaration**, **relation**) query has multiple true ancestors at varying hop distances. The legacy protocol also used edge-level random splits, which can fragment the same query’s positive set across train/val/test, and binary edge classification training (BCEWithLogitsLoss), which optimizes per-edge existence rather than within-query ranking quality.

Structural confounds. Real traced Lean/Mathlib hierarchy graphs are not uniform trees. The relation layer is often shallow and fragmented, with a high proportion of **instance_of** edges that are structurally flat—they register type-class instances without creating inheritance chains. Mixing these flat synthesized edges with genuine **extends** hierarchy edges may dilute the hierarchical signal that hyperbolic geometry is designed to exploit, but this composition effect has not been systematically controlled for.

1.4 Our Approach

We address these confounds through a staged experimental pipeline:

1. **Pipeline and protocol freeze** (Milestone 1): Version-lock the entire data-processing chain and freeze a grouped multi-positive ancestor retrieval protocol with query-level splits and hop-bucket analysis.
2. **Diagnostics and candidate selection** (Milestone 2): Characterize the structural properties of real formal-math graphs (longest chain, leaf ratio, component structure, multi-parent branching, cycle rank, delta-hyperbolicity proxy) and select candidates that span the range from shallow star forests to deeper hierarchies.
3. **Training alignment** (Milestone 3): Migrate from binary edge classification to query-grouped retrieval training (sampled softmax / InfoNCE-family loss), align training query keys to evaluation query keys (**src_id**, **relation_type**), and establish matched GCN/HGCN baselines on the full source graph.
4. **Provenance split** (Milestone 4): Systematically decompose the source graph into three provenance layers—**explicit_only** (extends edges alone), **synthesized_only** (**instance_of** edges alone), and **hierarchy_mixed** (both)—and repeat the matched comparison on each layer.
5. **Proof-side bridge** (Milestone 5): Demonstrate that the provenance-conditional finding has concrete implications for proof-engineering tool quality via an ancestor explanation CLI tool.

1.5 Central Claim

Edge provenance composition is a first-class experimental variable in formal-math hierarchy graph learning. On real traced Lean/Mathlib graphs, HGCN outperforms GCN only when the graph is restricted to explicit (**extends**) hierarchy edges; adding synthesized (**instance_of**) edges—which are structurally flat—dilutes the hierarchical signal enough to revert the comparison in GCN’s favor.

1.6 Non-Claims (Explicit Boundaries)

1. We do **not** claim that HGCN is generally superior to GCN on formal-math graphs.
2. We do **not** claim that the provenance-conditional finding generalizes beyond the two reviewed candidate graphs (Field.Subfield: 133 nodes, Order.Ring: 253 nodes).
3. We do **not** claim clean-room reproducibility has been fully closed; current evidence comes from reviewed single-environment runs.
4. We do **not** claim the pipeline covers full Mathlib or end-to-end theorem proving.
5. We do **not** claim the diagnostics thresholds are theoretically optimal; they are empirically calibrated heuristics from reviewed artifacts.

1.7 Contributions Summary

- **C1.** Reproducible traced formal-math hierarchy graph pipeline: version-locked from Lean/Mathlib traced repositories through declaration graphs, precise hierarchy extraction, coverage-aware repair, and relation-aware sub-graph construction.
- **C2.** Grouped multi-positive ancestor retrieval protocol: standardized evaluation with query-level split disjointness, MAP/nDCG/nDCG@10/MRR/Recall@k metrics, and hop-bucket analysis.
- **C3.** Graph structure diagnostics framework: heuristic thresholds for classifying formal-math graphs as shallow forest, star forest, or hierarchy-rich.
- **C4.** Provenance-conditional hyperbolic advantage finding: edge provenance composition—not graph size or model capacity—determines whether hyperbolic geometry helps.
- **C5.** Training task alignment correction: migration from binary edge classification to query-grouped retrieval training with aligned query keys.

2 Experimental Setup

2.1 Data: Traced Lean/Mathlib Hierarchy Graphs

Our pipeline extracts hierarchy graphs from traced Lean/Mathlib repositories. The processing chain consists of:

1. **Tracing:** Running LeanDojo on target repositories to produce normalized traces.
2. **Declaration graph construction:** Extracting nodes (declarations) and edges (`extends`, `instance_of`, `uses`) from trace data.
3. **Precise hierarchy extraction:** Filtering to `extends` and `instance_of` edges that form the hierarchy layer.
4. **Coverage-aware repair:** Handling unresolved endpoints—endpoints that can be backfilled are repaired; those that cannot are explicitly annotated and excluded from negative sampling.
5. **Relation-aware subgraph construction:** Extracting subgraphs centered on specific Mathlib modules (e.g., `Mathlib.Algebra.Field.Subfield`, `Mathlib.Algebra.Order.Ring`).

Candidate graphs. We select two candidate graphs for formal evaluation:

Table 1. Candidate graphs for formal evaluation.

Graph	Source Module	Nodes	Edges	Relations
Field.Subfield	<code>Mathlib.Algebra.Field.Subfield</code>	133	152	<code>extends</code> , <code>instance_of</code>
Order.Ring	<code>Mathlib.Algebra.Order.Ring</code>	253	300	<code>extends</code> , <code>instance_of</code>

These candidates were selected through the diagnostics framework (C3) to represent a range from a smaller, more controlled probe (Field.Subfield) to a larger, more balanced candidate (Order.Ring). Both contain only `extends` and `instance_of` edges (no `uses` edges in the hierarchy layer).

2.2 Provenance Split Design

We decompose each source graph into three provenance layers based on the origin of each edge:

Table 2. Provenance split design.

Split	Edges Included	Structural Role
<code>explicit_only</code>	<code>extends</code> only	Primary evidence: isolates genuine hierarchy
<code>synthesized_only</code>	<code>instance_of</code> only	Controlled diagnostic: confirms flat structure
<code>hierarchy_mixed</code>	Both	Reproducibility check: full source graph

The provenance split protocol is frozen with origin mapping: `extends` → `explicit`, `instance_of` → `synthesized`, both → `hierarchy_mixed`. The `hierarchy_mixed` = full source graph identity has been programmatically verified for both candidates.

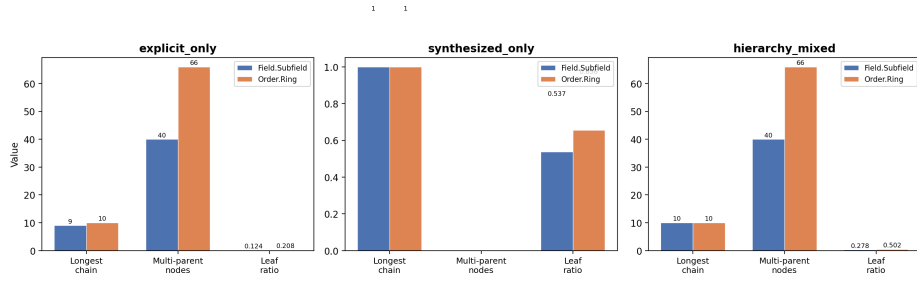


Fig. 1. Structural contrast across provenance splits. Left: **explicit_only** graphs exhibit genuine hierarchy depth (longest chain 9–10, multi-parent 40–66). Right: **synthesized_only** graphs are flat star forests (longest chain 1, multi-parent 0). Center: **hierarchy_mixed** inherits depth from explicit edges but gains leaf inflation from synthesized edges.

2.3 Structural Properties by Provenance Split

The provenance split produces dramatically different structural profiles (Figure 1):

Table 3. Structural properties of candidate graphs by provenance split. **synthesized_only** graphs are flat star forests (longest chain = 1, multi-parent = 0, cycle rank = 0). All hierarchy depth comes from **explicit_only** edges.

Property	Field.Subfield			Order.Ring		
	explicit	synth.	mixed	explicit	synth.	mixed
Longest chain	9	1	10	10	1	10
Multi-parent	40	0	40	66	0	66
Cycle rank	32	0	32	60	0	60
Leaf ratio	0.124	0.537	0.278	0.208	0.656	0.502
$\delta/\text{maxdist}$	0.286	0.000	—	0.136	0.000	—
Giant comp. ratio	0.506	—	—	0.792	—	—

Key observation: **synthesized_only** graphs are shallow star forests (longest chain = 1, multi-parent = 0, cycle rank = 0). All hierarchy depth comes from **explicit_only** edges. Adding synthesized edges to form **hierarchy_mixed** does not add meaningful depth—it inflates leaf ratio and fragmentation without contributing to hierarchy structure.

2.4 Evaluation Protocol

We use the **grouped multi-positive ancestor retrieval** protocol (C2):

- **Query:** (`src_id`, `relation_type`)—a declaration and a relation type.

- **Positives:** All true ancestors of `src_id` reachable via `extends` edges for the given relation type.
- **Split:** Query-level disjoint—all positives for the same `(src, relation)` query remain in the same split. Split ratios: `val_ratio = 0.15`, `test_ratio = 0.15`.
- **Metrics:** MAP, nDCG, nDCG@10, MRR, Recall@k.
- **Hop-bucket analysis:** Results decomposed by ancestor distance: `hop_2`, `hop_3`, `hop_4_plus`.

This protocol replaces the legacy single-positive `ancestor_ranking` approach, which treated ancestor retrieval as finding one positive among candidates, ignoring the multi-positive nature of the task.

2.5 Models and Training

We compare two models under matched conditions:

GCN (Euclidean baseline): Standard graph convolutional network operating in Euclidean space.

HGCN (Hyperbolic): Hyperbolic graph convolutional network with curvature parameter, residual connections, and distance signal integration (variant: `relation_hgcn_residual_v3`).

Matched training configuration:

Table 4. Matched training configuration for all experiments.

Parameter	Value
Dimensions (input/hidden/output)	16 / 16 / 16
Grouped loss	<code>sampled_softmax</code>
Negative ratio	10.0
Optimizer	SGD, <code>lr = 0.01</code>
Weight decay	<code>1e-4</code>
Epochs	100
Eval every	5 epochs
Patience	12
Seeds	[7, 42, 123, 2026, 3407]

Training uses query-grouped retrieval loss (C5) with query keys (`src_id`, `relation_type`) aligned to the evaluation protocol. Model selection uses validation grouped MAP.

2.6 Experimental Design

The experimental design consists of three phases:

1. **Milestone 3 baseline** (T32/T33): Matched GCN vs HGCN grouped 5-seed sweeps on `hierarchy_mixed` (full source graph). 20 runs (2 models $\times$ 2 graphs $\times$ 5 seeds, zero failures).
2. **Provenance-aware sweeps** (T42): Matched GCN vs HGCN grouped 5-seed sweeps on all three provenance splits. 60 runs total (2 models $\times$ 2 graphs $\times$ 3 splits $\times$ 5 seeds, zero failures).
3. **Proof-side bridge** (T52a): Ancestor explanation CLI tool that loads T42 reviewed node embeddings and performs provenance-aware ancestor retrieval ranking.

3 Results

3.1 Milestone 3 Baseline: Full Source Graph (`hierarchy_mixed`)

On the full source graph (equivalent to `hierarchy_mixed`), GCN outperforms HGCN on both candidates:

Table 5. Grouped retrieval results on `hierarchy_mixed` (full source graph). GCN vs HGCN, 5-seed mean $\pm$ std.

Graph	GCN MAP	HGCN MAP	Delta	GCN nDCG	HGCN nDCG
Field.Subfield	0.4839 ± 0.0783	0.4458 ± 0.1150	GCN +0.0381	0.6428 ± 0.0653	0.6095 ± 0.0908
Order.Ring	0.5789 ± 0.0346	0.5616 ± 0.0312	GCN +0.0173	0.7293 ± 0.0340	0.7111 ± 0.0296

Interpretation: Under the matched grouped protocol on the full source graph, GCN remains the stronger model. This is consistent with the original observation that hyperbolic inductive bias does not confer a general advantage on these graphs. However, this baseline does not control for edge provenance composition—the full graph mixes genuine hierarchy edges with flat synthesized edges.

3.2 Primary Evidence: `explicit_only`

On `explicit_only` graphs—which isolate genuine hierarchy structure—HGCN outperforms GCN on both candidates across all primary ranking metrics:

The Field.Subfield MAP improvement of +0.1247 is the largest model gap observed in the entire project. HGCN outperforms GCN on both candidates across all primary ranking metrics (MAP, nDCG, nDCG@10, MRR).

3.3 Hop-Bucket Analysis: Advantage Scales with Depth

HGCN’s advantage on `explicit_only` grows monotonically with hop depth on both candidates (Figure 2):

Table 6. Provenance-aware grouped retrieval results on `explicit_only` (primary evidence). 5-seed mean $\pm$ std.

Graph	Metric	GCN	HGCN	Delta
Field.Subfield	MAP	0.5256 ± 0.0800	0.6503 ± 0.0481	+0.1247
	nDCG	0.6864 ± 0.0691	0.7696 ± 0.0435	+0.0832
	nDCG@10	0.6002 ± 0.0888	0.6997 ± 0.0539	+0.0995
	MRR	0.5449 ± 0.1027	0.6738 ± 0.0588	+0.1289
Order.Ring	MAP	0.5836 ± 0.0978	0.6393 ± 0.0656	+0.0557
	nDCG	0.7332 ± 0.0725	0.7743 ± 0.0565	+0.0411
	nDCG@10	0.6224 ± 0.1347	0.7064 ± 0.0774	+0.0840
	MRR	0.5888 ± 0.1472	0.7211 ± 0.0902	+0.1323

Table 7. Hop-bucket MAP comparison on `explicit_only`. HGCN advantage grows monotonically with ancestor chain depth. 5-seed mean $\pm$ std. FS `hop_4_plus` based on 4/5 seeds (seed 2026 produces no `hop_4_plus` queries; symmetric between GCN and HGCN).

Graph	Bucket	GCN MAP	HGCN MAP	Delta
Field.Subfield	hop_2	0.3403 ± 0.1295	0.3946 ± 0.1402	+0.0543
	hop_3	0.2321 ± 0.1402	0.4050 ± 0.2321	+0.1729
	hop_4_plus [†]	0.2774 ± 0.1002	0.5245 ± 0.1419	+0.2471
Order.Ring	hop_2	0.2347 ± 0.0352	0.2615 ± 0.1066	+0.0268
	hop_3	0.2038 ± 0.0486	0.2989 ± 0.0958	+0.0951
	hop_4_plus	0.4506 ± 0.1582	0.7214 ± 0.0264	+0.2708

The HGCN advantage scales from approximately +0.03–0.05 at `hop_2` to approximately +0.25–0.27 at `hop_4_plus`. This monotonic growth is consistent with the theoretical motivation for hyperbolic geometry: the benefit comes from hyperbolic space’s ability to embed longer hierarchical paths with lower distortion, not from model capacity.

3.4 Controlled Diagnostic: `synthesized_only`

On `synthesized_only` graphs—which are structurally flat star forests—GCN matches or outperforms HGCN:

Table 8. Provenance-aware results on `synthesized_only` (controlled diagnostic).

Candidate	GCN MAP	HGCN MAP	Delta
Field.Subfield	1.0000 ± 0.0000	0.6857 ± 0.1140	GCN +0.3143
Order.Ring	0.8453 ± 0.0295	0.7560 ± 0.0761	GCN +0.0893

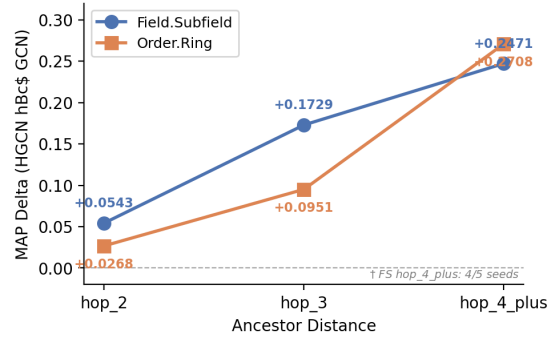


Fig. 2. HGCN vs GCN MAP delta on `explicit_only`, decomposed by hop depth. HGCN’s advantage grows monotonically from `hop_2` to `hop_4_plus` on both candidate graphs, confirming that hyperbolic geometry specifically benefits longer ancestor chains.

Note. Field.Subfield `synthesized_only` is a flat star forest (longest chain = 1, multi-parent = 0); GCN trivially achieves perfect MAP.

GCN matches or outperforms HGCN on the flat synthesized graphs. The hyperbolic inductive bias is a liability on structures with no hierarchy depth, confirming that HGCN’s advantage on `explicit_only` is driven by geometry matching the graph structure, not by model capacity.

3.5 Reproducibility Check: `hierarchy_mixed`

The `hierarchy_mixed` results from the provenance sweep (T42) byte-identically reproduce the Milestone 3 baseline (T32/T33):

Table 9. Byte-identical reproduction of Milestone 3 baseline by T42 provenance sweeps.

Sweep	T42 MAP	T32/T33 MAP Match
GCN Field.Subfield	0.4839 ± 0.0783	0.4839 ± 0.0783 exact
GCN Order.Ring	0.5789 ± 0.0346	0.5789 ± 0.0346 exact
HGCN Field.Subfield	0.4458 ± 0.1150	0.4458 ± 0.1150 exact
HGCN Order.Ring	0.5616 ± 0.0312	0.5616 ± 0.0312 exact

This byte-identical reproduction confirms both the `hierarchy_mixed` = full source graph identity and the internal consistency of the experimental pipeline.

3.6 Proof-Side Bridge: Ancestor Explanation

The ancestor explanation CLI tool demonstrates that the provenance-conditional finding has concrete implications for proof-engineering tool quality. The tool loads T42 reviewed node embeddings and performs ancestor retrieval ranking under different provenance modes.

Key demonstration: StrictOrderedCommRing on Order.Ring (HGCN, seed 42)

Table 10. Ancestor explanation quality comparison across provenance modes.

Mode	MAP	Recall@10	True ancestors in top-10
explicit_only	0.6438	0.1364	6/44
hierarchy_mixed	0.1492	0.0000	0/44

On `explicit_only`, HGCN retrieves true hierarchy ancestors (`AddCommGroup`, `SubNegMonoid`, `NonAssocSemiring`) in the top-10. On `hierarchy_mixed`, synthesized-instance nodes (`OrderDual.instAddCommGroupWithOne`, `CanonicallyOrderedAddCommMonoid`) crowd out all true ancestors from the top-10. This makes the aggregate provenance-conditional finding tangible at the individual-declaration level.

3.7 Summary: Provenance-Conditional Conclusion

Table 11. Provenance-conditional model comparison across all three splits. Edge provenance composition determines which geometry is favored.

Provenance Split	HGCN vs GCN (FS MAP)	HGCN vs GCN (OR MAP)	Structural Role
<code>explicit_only</code> (primary)	HGCN +0.1247	HGCN +0.0557	Deep hierarchy
<code>synthesized_only</code> (diagnostic)	GCN +0.3143	GCN +0.0893	Flat star forest
<code>hierarchy_mixed</code> (repro.)	GCN +0.0381	GCN +0.0173	Depth + leaf inflation

The question “does HGCN outperform GCN on formal-math graphs?” cannot be answered without specifying which provenance layer is being tested. Edge provenance composition is the decisive experimental variable.

4 Discussion

4.1 Why Provenance Matters

The provenance-conditional finding has both a structural explanation and an empirical confirmation:

Structural explanation. In Lean/Mathlib, `extends` edges encode genuine mathematical hierarchy—a `CommRing` extends a `Ring`, which extends a `Semiring`, creating chains of increasing abstraction. These chains have depth (up to 9–10 levels in our candidates), branching (40–66 multi-parent nodes), and genuine tree-like topology. In contrast, `instance_of` edges register type-class instances: they connect a concrete implementation to its abstract interface (e.g., `Int.instCommRing` is an instance of `CommRing`), but they do not create inheritance chains. The synthesized edges are structurally flat (longest chain = 1, multi-parent = 0).

Empirical confirmation. The hop-bucket analysis provides direct evidence that HGCN’s advantage scales with hierarchy depth: from +0.03–0.05 at `hop_2` to +0.25–0.27 at `hop_4_plus`. This is precisely what the hyperbolic geometry hypothesis predicts—longer chains benefit more from hyperbolic embedding—but it is confirmed only on `explicit_only`, not on the full graph.

4.2 The Compositional Artifact

The Milestone 3 conclusion “GCN overall ahead, HGCN not established as stronger” was correct for the full source graph. The provenance split reveals that this was a compositional artifact: the explicit hierarchy layer does favor HGCN, but this advantage is masked by the dominant flat synthesized layer in the mixed graph. Adding synthesized edges to the mixed graph:

- Does not increase longest chain (remains 9–10).
- Does not increase multi-parent count (remains 40–66).
- Increases leaf ratio from 0.12–0.21 to 0.28–0.50.
- Increases component count from 3–5 to 10–13.

The synthesized edges add structural noise—inflated leaf ratio and fragmentation—without contributing hierarchy depth. This noise is sufficient to negate HGCN’s advantage on the genuine hierarchy.

4.3 Implications for Future Work

1. **Provenance composition is a methodological requirement.** Future work on hierarchical graph learning in formal mathematics must control for edge provenance. Reporting model comparison results without specifying which provenance layer is being tested is incomplete.
2. **Hyperbolic advantage requires genuine depth.** The monotonic growth of HGCN’s advantage at deeper hops confirms that the benefit scales with hierarchy depth, not graph size.
3. **Diagnostics before deployment.** The structural diagnostics framework can predict whether hyperbolic models are worth testing on a given graph. Graphs with longest chain ≤ 2 , multi-parent count = 0, and high leaf ratio are unlikely to benefit from hyperbolic geometry.

4. **Training alignment is necessary but not sufficient.** The migration from binary edge classification to grouped retrieval training (C5) was necessary to produce valid model comparisons, but training alignment alone did not cause HGCN to outperform GCN. Provenance composition remains the decisive factor.
5. **Proof-side tools inherit provenance sensitivity.** The ancestor explanation demo shows that the quality of hierarchy navigation tools depends on which provenance layer the embeddings are trained on.

4.4 Related Work and Positioning

Hyperbolic embeddings and graph learning. Poincaré embeddings [4] initiated the use of hyperbolic geometry for representing hierarchical data, demonstrating exponential capacity gains for tree-like structures in low dimensions. HGCN [3] extended this to graph neural networks by performing message passing in hyperbolic space. Subsequent work explored Lorentzian models, hyperbolic attention mechanisms, and curvature-learning variants. Across this literature, evaluations typically use synthetic trees (WORDNET, Amazon) or biological taxonomies, where the hierarchical structure is unambiguous. Our work differs by applying hyperbolic graph learning to *engineered* hierarchies from proof assistants, where the structure is heterogeneous and the notion of “hierarchy” depends on which edge type is being considered.

Formal-math graph datasets and tooling. Prior graph-based approaches to formal mathematics include DeepMath/HOList [2] for premise selection in Isabelle, TacticToe [1] for Coq proof search, and LeanDojo [5] for traced Lean data extraction. These efforts focus on proof search or premise retrieval and treat the underlying graph as uniform. Our contribution is orthogonal: we provide infrastructure for *diagnosing* graph structure and systematically controlling for edge semantics before evaluating model choices.

Proof assistant hierarchy navigation. Interactive proof assistants maintain type-class hierarchies that are essential for understanding mathematical context. Lean’s `#print` commands and Mathlib’s documentation tools expose hierarchy relationships, but do not provide learned representations or ranking-based retrieval. Our ancestor explanation CLI bridges this gap by offering provenance-aware ranking over hierarchy ancestors, with the key insight that the *choice of which edges to include* determines retrieval quality.

Differentiation. This paper is not a “new model” contribution. Our contribution is methodological: we provide the pipeline, protocol, diagnostics, and provenance-conditional analysis needed to determine when hyperbolic geometry is worth deploying on formal-math graphs, and when it is not. The provenance-conditional finding itself—that edge provenance composition determines which geometry is favored—is a negative result for the blanket claim “hyperbolic always helps” but a positive methodological contribution for the community.

5 Limitations

5.1 Internal Validity

1. **Small graph scale.** Both reviewed candidates are small (133 and 253 nodes). The provenance-conditional finding is empirically established on these two graphs; statistical power for detecting interaction effects is limited. Generalization to larger formal-math graphs requires further evidence.
2. **Single-environment execution.** All 60 provenance sweep runs and all Milestone 3 baseline runs were executed in a single environment. Clean-room reproducibility has not been independently verified. We report our evidence as “reviewed single-environment runs,” not “independently reproduced.” Clean-environment reproduction remains an open item (R25).
3. **Hop_4_plus sample size.** On Field.Subfield `explicit_only`, `hop_4_plus` means are computed over 4 of 5 seeds (seed 2026 produces no `hop_4_plus` queries). The comparison is symmetric (both GCN and HGCN lack that seed), but statistical estimates at this bucket are less stable.
4. **Heuristic diagnostics thresholds.** The shallow-forest / star-forest / hierarchy-rich classification thresholds are empirically calibrated from current reviewed artifacts, not theoretically derived. They may not transfer to graphs with different size or topology distributions.

5.2 External Validity

5. **Lean/Mathlib specificity.** The provenance split semantics (`extends` = explicit, `instance_of` = synthesized) are Lean-specific. Other proof assistants (Coq, Isabelle) have different hierarchy mechanisms; the provenance-conditional finding may not directly transfer.
6. **Limited task scope.** This paper covers ancestor retrieval and parent prediction only. Other proof-side tasks (premise retrieval, declaration recommendation, proof search) may exhibit different geometry preferences.
7. **Module-level subgraph selection.** The two candidates are manually selected subgraphs from Mathlib. The selection itself introduces a form of reporting bias, even though selection criteria are documented and reviewed.

5.3 Scope Boundaries

8. **Page budget consideration (R30).** The five contributions (C1–C5) may be too many for the page limits of target venues (ITP and CPP typically allow ~ 20 pages). The current draft keeps C1–C5 as separate contributions with page-budget-aware wording. If further compression is needed, C3 or C5 can be condensed with full treatment deferred to the appendix; the core narrative (C1 $\rightarrow$ C2 $\rightarrow$ C4) and central claim remain self-contained.
9. **Proof-side bridge scope.** The ancestor explanation tool is a proof-of-concept demonstration, not an end-to-end theorem prover. It demonstrates that provenance-conditional quality differences are tangible, but does not claim that this translates directly to proof completion rates.

6 Conclusion

We have presented a reproducible pipeline, protocol, diagnostics framework, and training alignment correction for studying graph representation learning on real traced Lean/Mathlib hierarchy graphs. Our central finding is **provenance-conditional**: edge provenance composition—specifically whether the graph includes only genuine hierarchy edges (**extends**) or also flat synthesized edges (**instance_of**)—determines whether hyperbolic graph neural networks outperform Euclidean baselines.

On **explicit_only** graphs (genuine hierarchy), HGCN outperforms GCN by +0.1247 MAP (Field.Subfield) and +0.0557 MAP (Order.Ring), with the advantage growing monotonically with ancestor chain depth from +0.03 at `hop_2` to +0.27 at `hop_4_plus`. On **hierarchy_mixed** (full source graph), GCN remains ahead. On **synthesized_only** (flat star forests), GCN matches or outperforms HGCN.

This finding reframes the question from “which geometry wins?” to “which edge provenance composition determines which geometry is favored”—a methodological contribution for future work on hierarchical graph learning in formal mathematics.

Open Directions

- **Larger graph validation.** Testing the provenance-conditional finding on larger formal-math graphs (full Mathlib modules, multi-repository benchmarks).
- **Cross-assistant transfer.** Investigating whether analogous provenance splits exist in Coq and Isabelle hierarchy structures.
- **Broader task evaluation.** Extending beyond ancestor retrieval to premise retrieval, declaration recommendation, and proof search.
- **Clean-environment reproducibility.** Closing the R25 risk by independently reproducing the 60 provenance sweep runs in a fresh environment.
- **Artifact packaging.** Packaging the pipeline, protocol, diagnostics framework, and demo tool as a submittable CPP artifact bundle.

References

1. Gauthier, T., Kaliszyk, C., Urban, J.: TacticToe: Learning to reason with HOL4 tactics. In: Conference on Certified Programs and Proofs (CPP). ACM (2017)
2. Irving, G., Szegedy, C., Alemi, A.A., Eén, N., Chollet, F., Urban, J.: Deepmath - deep sequence models for premise selection. In: Advances in Neural Information Processing Systems (NeurIPS) (2016)
3. Liu, Q., Nickel, M., Kiela, D.: Hyperbolic graph neural networks. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 32 (2019)
4. Nickel, M., Kiela, D.: Poincaré embeddings for learning hierarchical representations. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 30 (2017)

5. Yang, K., Swope, A., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R.J., Anandkumar, A.: LeanDojo: Theorem proving with retrieval-augmented language models. In: Conference on Neural Information Processing Systems (NeurIPS) (2023)

A Evidence Chain

A.1 Reviewed Task Sequence

Table 12. Reviewed task sequence across five milestones.

Milestone	Tasks	Closure Review	Status
M0: Governance Bootstrap	T00–T02	T00 PASS, T01 PASS_W/W, T02 PM-accepted	Closed
M1: Data & Protocol Freeze	T10–T14	All PASS	Closed
M2: Diagnostics & Candidate Selection	T20–T22	T20 PASS_W/W, others PASS	Closed
M3: Grouped Retrieval Training Alignment	T30–T34	All PASS	Closed
M4: Relation Provenance Split	T40–T43	All PASS	Closed
M5: Paper & Proof-Side Bridge	T50–T52a, T53	T50 PASS_W/W, others PASS	Closed

Total tasks reviewed: 24 (plus T02 PM-accepted). Adversarial reviews: 11.

A.2 Key Numeric Anchors (Reviewed, Frozen)

All numeric values in this paper come from reviewed T32/T33/T42/T43 artifacts. They must not be altered in subsequent drafts without re-running the corresponding experiments.

Table 13. Key numeric anchors from reviewed artifacts.

Quantity	Value	Source
GCN MAP, FS <code>hierarchy_mixed</code>	0.4839 ± 0.0783	T32/T42 (exact)
HGCN MAP, FS <code>hierarchy_mixed</code>	0.4458 ± 0.1150	T33/T42 (exact)
GCN MAP, OR <code>hierarchy_mixed</code>	0.5789 ± 0.0346	T32/T42 (exact)
HGCN MAP, OR <code>hierarchy_mixed</code>	0.5616 ± 0.0312	T33/T42 (exact)
GCN MAP, FS <code>explicit_only</code>	0.5256 ± 0.0800	T42
HGCN MAP, FS <code>explicit_only</code>	0.6503 ± 0.0481	T42
GCN MAP, OR <code>explicit_only</code>	0.5836 ± 0.0978	T42
HGCN MAP, OR <code>explicit_only</code>	0.6393 ± 0.0656	T42
FS explicit hop_4_plus delta	+0.2471 (4/5 seeds)	T42
OR explicit hop_4_plus delta	+0.2708 (5/5 seeds)	T42
FS <code>synthesized_only</code> GCN MAP	1.0000 ± 0.0000	T42 (T56 verified)
FS <code>synthesized_only</code> HGCN MAP	0.6857 ± 0.1140	T42
OR <code>synthesized_only</code> GCN MAP	0.8453 ± 0.0295	T42
OR <code>synthesized_only</code> HGCN MAP	0.7560 ± 0.0761	T42